Save Point

The web app

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

`webapp/` is QuillFin in the browser: Next.js 16 (App Router), React 19, Tailwind 4, Clerk for sign-in and the Convex React client for data. It uses the same account and the same data as the Flutter app. A note written on the web reaches your phone on its next sync, and the other way round.

Unlike the Flutter app, the web app is online-only. It reads Convex directly, with live queries, so there's no local copy and no sync engine. Every write still goes through the backend's `RowWriter` (`backend/convex/rows.ts`), so the apps pull web edits like any other change (`backend.md`).

Running it

```
npm install                                  # repo root
cp webapp/.env.example webapp/.env.local     # then fill it in
npm run web:dev                              # http://localhost:3000
npm run web:test                             # 31 tests
npm run web:build
```

`.env.local` needs `NEXT_PUBLIC_CONVEX_URL`, `NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY` and `CLERK_SECRET_KEY` (`setup_guide.md` §7). Without real Clerk keys the build still succeeds (CI builds with placeholders), but signing in can't work.

Layout

```
webapp/
  app/
    layout.tsx          fonts (Outfit, Inter via next/font), theme boot script, Clerk +
Convex providers
    brand.css           the brand tokens, light and dark (the other copy is
lib/theme/brand.dart)
    globals.css         Tailwind, the breakpoints, token names
    page.tsx            landing: create an account / sign in (signed-in users go to /today)
    sign-in/, sign-up/  Clerk's components
    (app)/              everything behind sign-in, in the AppShell
```

```
    today/ notes/ quiz/ tracker/ settings/ account/
    settings/plan/     where Stripe returns to (redirects to Settings > Plan)
proxy.ts               Clerk: every page but the landing and sign-in needs an account
components/            the screens and shared UI
lib/                   the API, error messages, time helpers, per-browser preferences
test/                  vitest + Testing Library, with Convex and Next navigation mocked
```

The Convex API types and the shared logic come from the backend workspace (`@quillfin/backend` , transpiled by Next):

- `convex/_generated/api` : every function, typed;
- `convex/scheduling.ts` : FSRS, a direct port of the Dart scheduler, used for the rating buttons' previews (the server schedules the actual review with the same code);
- `convex/grading.ts` : short-answer grading, so the result shows instantly.

Screens

Breakpoints match the app's window size classes (`quillfin_plan.md` §5): Tailwind's `sm` , `md` , `lg` and `xl` are remapped to 600, 840, 1200 and 1600.

Screen	Phone	Wider
Shell	bottom bar	rail from 600; labelled sidebar from 1200
Today	hero, 2×2 stats, 7-day forecast, problems due	two columns from 840
Notes	folders → folder → note, one at a time	folders and notes left, editor right, from 840
Quiz	one column, rating buttons pinned to the bottom	the same, max 640 px, buttons under the answer
Tracker	filter chips, cards, detail full-screen	list and detail from 840; table view from 1200
Settings	one list	section list and section from 840

- **Keyboard**: Ctrl/⌘ + 1-5 switch sections, Ctrl/⌘ + S saves a note, Enter submits an answer, and 1-4 rate it.
- **Notes**: folders and subfolders, notes, questions (edit, pause, delete), and **Generate** on QuillFin's AI within your plan. Your own API key only works in the app.
- **Quiz**: takes what's due when it starts, so questions don't shift as reviews are recorded. Grading and suggested ratings match the app exactly.
- **Tracker**: search, filters, sortable table, detail with solution, notes and attempts, and Log attempt with FSRS previews.
- **Settings**: Plan (live: it updates by itself when Stripe's webhook lands), Appearance (System / Light / Dark), and target recall. The last two are kept per browser, as the app keeps them per device.
- **Account**: Clerk's account page (profile, password, sessions, delete).

Selections live in the URL (`/notes?folder=…&note=…` , `/tracker?p=…`), so back and forward and shared links work.

Testing

`test/` uses Vitest with jsdom and Testing Library. `test/setup.ts` replaces `convex/react` with a registry (`test/mocks.ts`): queries answer with canned results by function name (`answer("notes:get", …)`), and mutation and action calls are recorded (`calls("questions:record")`) or handled (`handle("ai:generateQuestions", …)`). The tests drive the real components:

- the quiz flow, including grading, interval previews, keyboard rating and errors;
- notes and folders, saving, generating and editing questions;
- the tracker's filters, table, detail, add, edit and logging attempts;
- the plan's checkout and portal, settings, and the shell's shortcuts.

Coverage of `components/` and `lib/` is about 95% of lines. The page files are thin wrappers checked by `next build`. Nothing here runs against real Clerk or Convex; `setup_guide.md` §7 lists the checks for the first real deploy.